#7 행정구역 초기 적재 — 코드리뷰 대비 설명서

출처: Notion `2차프로젝트 → [#7] 코드리뷰 · 행정구역 초기 적재 (PR #100)` (개정판 기준)

1. 한 문장 요약

> 전국 읍·면·동의 지도 경계 파일(GeoJSON)을 읽어서, 시·도 → 시·군·구 → 읍·면·동 소속 관계까지 정리해 DB의 `region` 표에 저장하는 **1회성 작업**이다.

왜 필요한가? 뒤에서 만드는 기능(#9 구역 미리보기, #10 구역 등록, #11 캠 등록)이 "이 구역은 어느 동에 속하나?"를 알아야 한다. 그 기준 데이터가 이 표다. #7이 없으면 뒤 기능이 전부 막힌다.

비유 (택배 물류센터)

- 경계 파일 = 택배 상자
- `RegionLoadJobRunner` = 경비원. 작업 지시서(`--job`, `--file`)가 맞는지 확인
- `RegionLoadService` = 센터장. 전체를 지휘
- `RegionGeoJsonParser` = 검수원. 상자를 뜯어서 검사
- `RegionRepository` = 창고 직원. 선반(DB)에 넣기

```
경계 파일 → Runner(--job, --file 확인) → Service(전부 되거나 전부 취소)
                                          ├→ Parser(검사·묶기·합치기) → RegionSource 목록
                                          └→ Repository → DB region 표
```

2. 개정판이 나온 이유 (선배가 제일 먼저 물을 부분)

처음에는 "코드가 2자리(시) / 5자리(구) / 7자리(동)이고, 파일에 시·구·동이 다 들어 있다"고 **가정**하고 만들었다. leads가 실제 파일(admdongkor)을 확인해 보니 달랐다.

구분	내가 가정한 것	실제 파일
코드 자릿수	2 / 5 / 7	**8자리** (`adm_cd`)
파일 내용	시·구·동 다 있음	**읍·면·동 다각형만** 있음

비유: 전교생 명단에 학년·반·학생이 다 있다고 생각했는데, 받아 보니 학생 명단뿐이었다. 그래서 학생 번호 **앞 2칸이 같으면 같은 학년(시·도), 앞 5칸이 같으면 같은 반(시·군·구)** 으로 묶어서 학년 명단과 반 명단을 우리가 직접 만든다. 학년 지도와 반 지도도 학생들 지도를 **이어 붙여서(union)** 만든다.

#	leads 리뷰	반영
1	`adm_cd`는 8자리	`EMD_CODE_LENGTH = 8`, 8자리 숫자가 아니면 중단
2	읍·면·동만 있으니 상위는 코드 앞자리로 만들고 경계는 JTS로 합침	`UnaryUnionOp.union` 사용
3	`RegionLevel`을 `SIDO/SIGUNGU/EMD`로	enum 이름 변경

#	leads 리뷰	반영
4	경도 ±180, 위도 ±90 밖이면 에러	`toRing`에서 범위 검사
5	실제 파일로 적재 결과를 남기기	대전 중구·수원 영통구·김포시 43개로 실행, 48건 확인

3. 파일별 코드와 설명

3-1. DB 표: V2__create_region.sql

```
CREATE TABLE region (
    id BIGINT NOT NULL AUTO_INCREMENT COMMENT '행정구역 번호',
    parent_id BIGINT NULL COMMENT '상위 행정구역(region.id). 시는 NULL',
    level VARCHAR(10) NOT NULL COMMENT '단위: SIDO / SIGUNGU / EMD',
    code VARCHAR(10) NOT NULL COMMENT '행정구역 코드(경계 데이터 원본 코드)',
    name VARCHAR(50) NOT NULL COMMENT '행정구역 이름',
    boundary JSON NOT NULL COMMENT '경계 다각형(GeoJSON, 위도·경도 WGS84)',
    center_lat DECIMAL(9, 6) NOT NULL COMMENT '경계 중심점 위도(경계 안의 점)',
    center_lng DECIMAL(9, 6) NOT NULL COMMENT '경계 중심점 경도',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '초기 데이터 적재 시각',
    CONSTRAINT region_pk PRIMARY KEY (id),
    CONSTRAINT region_uk_code UNIQUE (code),
    CONSTRAINT region_fk_parent FOREIGN KEY (parent_id) REFERENCES region (id)
) ENGINE = InnoDB;

CREATE INDEX region_ix_parent ON region (parent_id);
```

무슨 기능인가? DB에 엑셀 시트 하나(`region`)를 만든다. Flyway가 `V2__`라는 이름을 보고 "2번째 DB 변경"으로 자동 실행한다. 기존 V1은 건드리지 않고 V2를 추가했다.

DB가 직접 지켜주는 규칙 3개

- `UNIQUE (code)` : 같은 코드는 두 줄 못 들어간다. 사직동이 두 번 저장되는 사고를 DB가 막는다.
- `FOREIGN KEY (parent_id)` : 부모 칸에는 **실제로 있는 지역만** 넣을 수 있다.
- `INDEX (parent_id)` : "이 구에 속한 동 전부"를 빨리 찾게 해주는 목차.

`level` 칼럼 구조는 그대로고 주석만 `SIDO/SIGUNGU/EMD`로 바꿨다. `VARCHAR(10)`에 `SIGUNGU`(7글자)가 들어가므로 길이는 충분하다.

3-2. 표의 Java 버전: Region.java (common 모듈)

```
@Entity
@Table(name = "region")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Region {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "parent_id")
    private Region parent;

    @Enumerated(EnumType.STRING)
```

```
    @Column(nullable = false, length = 10)
    private RegionLevel level;

    @Column(nullable = false, length = 10, unique = true)
    private String code;

    @Column(nullable = false, length = 50)
    private String name;

    @JdbcTypeCode(SqlTypes.JSON)
    @Column(nullable = false)
    private String boundary;

    @Column(name = "center_lat", nullable = false, precision = 9, scale = 6)
    private BigDecimal centerLat;

    @Column(name = "center_lng", nullable = false, precision = 9, scale = 6)
    private BigDecimal centerLng;

    // DB 기본값(CURRENT_TIMESTAMP)이 채운다. 다시 적재해도 처음 적재 시각을 유지한다.
    @Generated(event = EventType.INSERT)
    @Column(name = "created_at", insertable = false, updatable = false)
    private LocalDateTime createdAt;

    public Region(Region parent, RegionLevel level, String code, String name,
                  String boundary, BigDecimal centerLat, BigDecimal centerLng) { ... }

    /** 경계 파일을 다시 적재할 때 같은 code의 행을 새 값으로 덮어쓴다(FNC-A-012). */
    public void overwrite(Region parent, RegionLevel level, String name,
                          String boundary, BigDecimal centerLat, BigDecimal centerLng) { ... }
}
```

SQL 표의 **한 줄 = Java 객체 하나**다.

질문	답
왜 `common`에 뒀나?	관리자 앱뿐 아니라 **사용자 앱도 나중에 지역 정보를 읽어야** 해서.
`LAZY`는 왜?	동 하나 읽을 때 구·시까지 줄줄이 불러오면 낭비라서, 부모는 **필요할 때만** 가져온다.
`@Enumerated(STRING)`은 왜?	글자(`EMD`)로 저장한다. 숫자 저장이면 enum 순서가 바뀔 때 의미가 꼬인다.
setter가 없는 이유는?	값을 바꾸는 길을 `overwrite()` 하나로 막아 **어디서 바뀌는지 추적**하기 쉽다(팀 컨벤션).
`overwrite`에 `code`가 없는 이유는?	`code`는 그 지역의 주민등록번호라 바꾸지 않는다.
`created_at`을 `insertable=false`로 둔 이유는?	시각은 DB만 정하게 해서, 다시 적재해도 **처음 적재 시각이 유지**된다.
`@NoArgsConstructor(PROTECTED)`는?	JPA가 객체를 만들 때 빈 생성자가 필요하고, `PROTECTED`라 밖에서 함부로 못 쓴다.

3-3. RegionLevel, RegionRepository

```
public enum RegionLevel {
    SIDO,      // 시·도
    SIGUNGU,   // 시·군·구
    EMD        // 읍·면·동
}
```

```
public interface RegionRepository extends JpaRepository<Region, Long> {
    List<Region> findByCodeIn(Collection<String> codes);
}
```

- enum은 "단위는 이 세 개뿐"이라고 못 박은 것이다. `EMDD` 같은 오타는 컴파일에서 바로 걸린다.
- **선언 순서가 곧 큰 단위 → 작은 단위 순서**다. Service가 이 순서로 정렬해서 저장하므로 순서를 바꾸면 저장이 깨진다.
- `findByCodeIn`은 **메서드 이름이 곧 명령**이다. `findBy` + `Code` + `In`이라서 스프링이 `WHERE code IN (...)`을 자동으로 만든다.
- 이 메서드는 "이미 DB에 있는 지역"을 **한 번에** 찾으려고 만들었다. 지역마다 DB를 수천 번 부르지 않으려는 것이다.

3-4. 메모지와 경보: RegionSource, RegionFileFormatException

```
/** 경계 파일의 지역 하나. parentCode는 시일 때 null. */
public record RegionSource(
        String code, String parentCode, RegionLevel level,
        String name, String boundary,
        BigDecimal centerLat, BigDecimal centerLng) {}
```

```
public class RegionFileFormatException extends RuntimeException {
    public RegionFileFormatException(String message) { super(message); }
    public RegionFileFormatException(String message, Throwable cause) { super(message, cause); }
}
```

- `RegionSource`는 검수원(Parser)이 센터장(Service)에게 넘기는 **정리된 메모지**다. `record`라서 한 번 만들면 못 바꾼다. 전달 중에 값이 바뀔 일이 없다.
- `RegionFileFormatException`은 "파일이 이상해요" 전용 에러다. 그냥 `RuntimeException`이면 "DB 문제인지 파일 문제인지" 구분이 안 된다.
- `RuntimeException`을 상속했기 때문에 `@Transactional` 안에서 이 에러가 나면 스프링이 **자동으로 롤백**한다. 중요한 연결고리다.

3-5. 검수원: RegionGeoJsonParser (개정판, 핵심 ①)

(a) 상수

```
private static final String CODE_PROPERTY = "adm_cd";
private static final String NAME_PROPERTY = "adm_nm";
private static final String SIDO_NAME_PROPERTY = "sidonm";
private static final String SIGUNGU_NAME_PROPERTY = "sggnm";
private static final int SIDO_CODE_LENGTH = 2;
private static final int SIGUNGU_CODE_LENGTH = 5;
private static final int EMD_CODE_LENGTH = 8;
private static final int MAX_NAME_LENGTH = 50;
private static final int COORDINATE_SCALE = 6;
private static final double MAX_LONGITUDE = 180;
private static final double MAX_LATITUDE = 90;
```

숫자와 글자를 코드 곳곳에 직접 쓰면 오타가 나고 의미를 알 수 없어서, 이름을 붙여 한 곳에 모았다(매직 넘버 금지 규칙).

(b) parse(): 파일 전체 검사

```
public List<RegionSource> parse(String geoJson) {
    final JsonNode root;
    try {
        root = jsonMapper.readTree(geoJson);
```

```
    } catch (JacksonException e) {
        throw new RegionFileFormatException("GeoJSON을 읽을 수 없습니다", e);
    }
    if (!"FeatureCollection".equals(root.path("type").asString(null))
            || !root.path("features").isArray()) {
        throw new RegionFileFormatException("FeatureCollection 형식이 아닙니다");
    }

    List<Emd> emds = new ArrayList<>();
    Set<String> codes = new HashSet<>();
    for (JsonNode feature : root.path("features")) {
        Emd emd = toEmd(feature);
        if (!codes.add(emd.code())) {      // Set은 같은 값을 못 넣음 → false면 중복
            throw new RegionFileFormatException("행정구역 코드가 중복되었습니다: " + emd.code());
        }
        emds.add(emd);
    }
    if (emds.isEmpty()) {
        throw new RegionFileFormatException("행정구역이 하나도 없습니다");
    }
    return toSources(emds);
}
```

검사 순서: ① JSON으로 읽히나 → ② `FeatureCollection`인가 → ③ 읍·면·동을 하나씩 읽기 → ④ 코드 중복 → ⑤ 비어 있지 않나. 하나라도 틀리면 **파일 전체를 거부**한다.

(c) toEmd(): 읍·면·동 하나 검사

```
String code = properties.path(CODE_PROPERTY).asString("").trim();
if (code.length() != EMD_CODE_LENGTH || !code.chars().allMatch(Character::isDigit)) {
    throw new RegionFileFormatException(CODE_PROPERTY + " 값은 8자리 숫자여야 합니다: " + code);
}
String fullName = properties.path(NAME_PROPERTY).asString("").trim();
...
String name = checkedName(fullName.substring(fullName.lastIndexOf(' ') + 1), code);
String sidoName = checkedName(properties.path(SIDO_NAME_PROPERTY).asString("").trim(), code);
String sigunguName = checkedName(properties.path(SIGUNGU_NAME_PROPERTY).asString("").trim(), code);
```

- 코드가 8자리 숫자인지 확인한다.
- 이름은 `adm_nm`의 **마지막 단어**를 쓴다. "서울특별시 종로구 사직동"에서 "사직동"만 가져온다.
- 시·도 이름은 `sidonm`, 시·군·구 이름은 `sggnm`에서 가져온다(leads 리뷰 2번).

(d) toSources(): 묶기 (이번 개정의 핵심)

```
Map<String, Group> sidos = new LinkedHashMap<>();
Map<String, Group> sigungus = new LinkedHashMap<>();
for (Emd emd : emds) {
    group(sidos, emd.code().substring(0, SIDO_CODE_LENGTH), emd.sidoName())
            .geometries.add(emd.geometry());
    group(sigungus, emd.code().substring(0, SIGUNGU_CODE_LENGTH), emd.sigunguName())
            .geometries.add(emd.geometry());
}

List<RegionSource> sources = new ArrayList<>();
sidos.forEach((code, group) -> sources.add(toSource(code, null, RegionLevel.SIDO, group)));
sigungus.forEach((code, group) ->
        sources.add(toSource(code, code.substring(0, SIDO_CODE_LENGTH), RegionLevel.SIGUNGU, grou
p)));
for (Emd emd : emds) {
    ... sources.add(new RegionSource(emd.code(), emd.code().substring(0, SIGUNGU_CODE_LENGTH),
```

```
                                    RegionLevel.EMD, ...));
}
```

- 읍·면·동 하나를 읽을 때마다, 코드 **앞 2자리 통(시·도)** 과 **앞 5자리 통(시·군·구)** 에도 그 도형을 넣는다.
- 목록은 **시·도 → 시·군·구 → 읍·면·동** 순서로 만든다. 부모가 먼저 저장돼야 하기 때문이다.
- 읍·면·동의 부모 코드는 앞 5자리, 시·군·구의 부모 코드는 앞 2자리다.
- 부모가 코드에서 만들어지므로 "부모가 파일에 있는지" 검사(`validateParents`)가 필요 없어져 **삭제**했다.

(e) group(): 이름 충돌 검사

```
Group group = groups.computeIfAbsent(code, key -> new Group(name));
if (!group.name.equals(name)) {
    throw new RegionFileFormatException(
        "같은 코드(" + code + ")에 이름이 둘 이상입니다: " + group.name + ", " + name);
}
```

같은 시·군·구 코드(`11110`)인데 한 줄만 이름이 다르면, 파일이 이상하다고 보고 중단한다.

(f) union(): 지도 이어 붙이기

```
private Geometry union(List<Geometry> geometries, String code) {
    try {
        return UnaryUnionOp.union(geometries);
    } catch (TopologyException e) {
        // 경계선이 꼬인 원본 도형이 섞여 있으면 buffer(0)으로 바로잡아 다시 합친다
        try {
            return UnaryUnionOp.union(geometries.stream().map(g -> g.buffer(0)).toList());
        } catch (TopologyException retry) {
            throw new RegionFileFormatException("경계를 합칠 수 없습니다: " + code, retry);
        }
    }
}
```

- 종로구 지도는 사직동 + 삼청동 지도를 **풀로 붙인** 것이다. JTS의 `UnaryUnionOp`가 이 일을 한다.
- 원본 선이 꼬여 있으면 합치다 실패할 수 있어서, `buffer(0)`으로 매만져 한 번 더 시도한다.
- 그래도 안 되면 우리 에러로 바꿔서 던진다.
- `buffer(0)` 경로를 실제 파일에서 타 봤는지는 확인하지 못했다. 실행 중 예외 없이 통과했을 뿐이다.

(g) toRing(): 좌표 검사 (리뷰 4번)

```
// GeoJSON 순서는 [경도, 위도]
double longitude = position.get(0).asDouble();
double latitude = position.get(1).asDouble();
// 좌표계가 다른 파일이면 DB 저장 때 원인을 알기 어려우므로 여기서 먼저 알린다
if (Math.abs(longitude) > MAX_LONGITUDE || Math.abs(latitude) > MAX_LATITUDE) {
    throw new RegionFileFormatException("위도·경도 범위를 벗어난 좌표입니다(WGS84 경도 ±180, 위도 ±90): "
            + code + " [" + longitude + ", " + latitude + "]");
}
coordinates[i] = new Coordinate(longitude, latitude);
```

- 좌표계가 다른 파일(예: 값이 955156 같은 평면 좌표)을 그대로 넣으면 DB의 `DECIMAL(9,6)`에서 이유를 알 수 없는 에러만 난다. 그래서 **여기서 먼저** 원인을 알려준다.
- GeoJSON은 **[경도, 위도]** 순서다. 평소 말하는 "위도, 경도"와 반대라 헷갈리기 쉽다. 선배가 물어볼 수 있는 포인트다.

(h) toGeoJson(): 합친 결과를 글자로 직접 변환

JTS의 변환 도구(`jts-io-common`)는 프로젝트에 없다. 추가하려면 `build.gradle`을 건드려야 하고 그 파일은 leads 리뷰 대상이다. 그래서 **직접 써서 의존성을 늘리지 않았다**. `Polygon`과 `MultiPolygon`만 지원하고 다른 도형이면 에러를 던진다.

(i) 가장 똑똑한 부분: getInteriorPoint()

```
Point interiorPoint = merged.getInteriorPoint();
... scaled(interiorPoint.getY()), scaled(interiorPoint.getX())
```

지도에 "삼청동" 핀을 꽂으려면 점 하나가 필요하다. 그냥 **무게중심**을 쓰면 문제가 생긴다.

- **도넛 모양** 동네는 무게중심이 가운데 **구멍**에 떨어진다.
- **섬이 두 개인** 동네는 무게중심이 두 섬 사이 **바다**에 떨어진다.

`getInteriorPoint()`는 **반드시 도형 안쪽에 있는 점**을 보장한다. 그래서 샘플에 일부러 구멍 있는 삼청동과 조각 두 개짜리 사직동을 넣고 테스트로 확인한다. 이 부분은 명세(FNC-A-012)가 요구한 것이다.

`scaled()`는 소수점 6자리로 반올림한다. DB 칸 `DECIMAL(9,6)`에 맞추려는 것이다.

3-6. 센터장: RegionLoadService (핵심 ②)

```
@Service
public class RegionLoadService {

    @Transactional
    public int load(Path file) {
        List<RegionSource> sources = parser.parse(read(file));      // ① 파일 전체를 먼저 검사

        // 상위(시·도 → 시·군·구 → 읍·면·동)부터 저장해야 parent_id를 연결할 수 있다
        List<RegionSource> ordered = sources.stream()
                .sorted(Comparator.comparing(RegionSource::level))   // ② enum 선언 순서로 정렬
                .toList();
        Map<String, Region> existing = regionRepository
                .findByCodeIn(ordered.stream().map(RegionSource::code).toList())   // ③ 기존 것 한 번에
조회
                .stream()
                .collect(Collectors.toMap(Region::getCode, region -> region));
        Map<String, Region> loaded = new HashMap<>();

        for (RegionSource source : ordered) {
            Region parent = source.parentCode() == null ? null : loaded.get(source.parentCode());
            Region region = existing.get(source.code());
            if (region == null) {
                region = regionRepository.save(new Region(parent, source.level(), source.code(),
                        source.name(), source.boundary(), source.centerLat(), source.centerLng()));
            } else {
                region.overwrite(parent, source.level(), source.name(),
                        source.boundary(), source.centerLat(), source.centerLng());
            }
            loaded.put(source.code(), region);
        }
        ...
        return ordered.size();
    }
}
```

1. **검사 먼저, 저장 나중**: `parser.parse()`가 파일을 다 검사한 뒤에야 저장을 시작한다. 불량 파일은 DB에 손도 못 댄다.

2. **`@Transactional` = 게임의 세이브 포인트**: 저장 도중 에러가 나면 시작 전 상태로 되감긴다. 1000개 중 500개만 저장된 반쪽 상태가 안 생긴다. 위의 "검사 먼저"와 합치면 **이중 안전장치**다.
3. **왜 큰 단위부터 저장하나?** 부모가 먼저 DB에 있어야 자식이 가리킬 수 있다. 태어나지 않은 사람을 아빠로 적을 수는 없다.
4. **`existing` 과 `loaded` 두 사전**: `existing`은 DB에 원래 있던 것, `loaded`는 이번에 처리 끝난 것이다. 자식이 `loaded`에서 부모를 찾는다.
5. **멱등성**: 몇 번을 돌려도 결과가 같다. 엘리베이터 버튼을 10번 눌러도 한 대만 오는 것과 같다. DB에 있으면 `overwrite`, 없으면 `save`다. 새 경계 파일이 나오면 그냥 다시 돌리면 된다.
6. **더티 체킹**: `overwrite`만 호출하고 `save`를 안 불러도, 트랜잭션 안에서 JPA가 변경을 알아채 자동으로 UPDATE한다.

3-7. 경비원: RegionLoadJobRunner

```
@Component
@ConditionalOnProperty(name = "job", havingValue = RegionLoadJobRunner.JOB_NAME)
public class RegionLoadJobRunner implements ApplicationRunner {

    static final String JOB_NAME = "load-regions";
    private static final String FILE_OPTION = "file";

    @Override
    public void run(ApplicationArguments args) {
        List<String> files = args.getOptionValues(FILE_OPTION);
        if (files == null || files.size() != 1) {
            throw new IllegalArgumentException("--file={경계 파일} 옵션이 하나 필요합니다");
        }

        int loaded = regionLoadService.load(Path.of(files.get(0)));
        log.info("행정구역 적재 작업 종료 건수={}", loaded);

        // 1회 실행 작업이므로 웹 서버를 계속 띄워 두지 않는다.
        // 실패하면 예외로 시작이 중단되어 종료 코드가 0이 아니다.
        System.exit(SpringApplication.exit(context));
    }
}
```

- 실행 방법: `--job=load-regions --file=경계파일.geojson`
- `@ConditionalOnProperty`는 `--job=load-regions`가 있을 때**만** 이 클래스를 만든다. 평소 관리자 앱 운영에는 영향이 없다.
- 일이 끝나면 스스로 종료한다. 이 작업은 한 번 하고 끝나는 일인데 관리자 앱은 원래 웹 서버라 안 끄면 계속 켜져 있기 때문이다.
- 실패하면 예외로 시작이 중단돼서 종료 코드가 0이 아니다. 자동화 스크립트가 "실패했구나"를 알 수 있다.

3-8. admin/build.gradle

```
// 다각형 계산: 내부 점(getInteriorPoint)
implementation 'org.locationtech.jts:jts-core:1.20.0'
```

JTS(도형 계산 라이브러리)를 **admin 모듈에만** 추가했다. 사용자 앱에는 필요 없으니까. 이 파일은 CODEOWNERS(leads) 리뷰 대상이다.

3-9. 테스트 10개 (전부 통과)

테스트	증명하는 것
`loadTwice_noDuplicates`	두 번 돌려도 6건, **id도 그대로**여서 덮어쓰기라는 증거
`load_linksParentByCode`	`11` → `11110` → `11110530` 사직동으로 소속이 이어짐
`load_centerIsInsideBoundary`	삼청동 중심점이 구멍 안에 떨어지지 않음
`load_overwritesSameCode`	"삼청동"을 "새삼청동"으로 바꾼 파일이 이름만 바꿈
`load_invalidFile_savesNothing`	도형 종류가 이상하면 에러, DB는 0건
`load_notJson_fails`	JSON이 아니면 중단
`load_mergesBoundaryOfChildren` (신규)	종로구 경계는 사직동+삼청동 범위, 합쳐도 삼청동 구멍은 남음
`load_codeNotEightDigits_fails` (신규)	7자리 코드면 중단, 0건 저장
`load_coordinateOutOfRange_fails` (신규)	경도 955156.5 같은 값이면 "범위를 벗어난" 메시지로 중단
`load_conflictingSigunguName_fails` (신규)	같은 시·군·구 코드에 이름이 다르면 중단

테스트 샘플은 서울 종로구(사직동은 MultiPolygon, 삼청동은 구멍 있음)와 중구(소공동), 총 3개 읍·면·동이다. 시·도 1 + 시·군·구 2 + 읍·면·동 3 = **6건**이 저장된다.

구성의 장점은 "잘 되는 경우"와 "망가진 경우"를 둘 다 본다는 것이다. 망가진 경우는 모두 "에러가 나고 + DB는 0건"까지 확인해서 **전부 아니면 전무**를 증명한다. "상위 없음" 테스트는 삭제했다. 상위를 이제 파일이 아니라 코드에서 만들어서 그런 상황이 일어날 수 없기 때문이다.

실제 파일(전국 3,559개 중 43개를 잘라서)로도 돌렸다. 첫 실행 48건, 두 번째 실행도 48건이라 중복이 없었다.

4. 선배가 물을 만한 질문과 답

질문	답
"그냥 파일 읽어서 DB에 넣으면 되지, 왜 이렇게 복잡해?"	① 검사 먼저 + 트랜잭션이라 반쪽 데이터가 안 생긴다. ② code 기준 덮어쓰기 + UNIQUE라 몇 번 돌려도 안전하다. ③ 부모-자식이 코드 규칙 + FK + 저장 순서로 이중 삼중 보호된다. ④ 무게중심 대신 내부 점이라 핀이 엉뚱한 곳에 안 꽂힌다. ⑤ `--job`을 줄 때만 켜지고 끝나면 꺼진다. ⑥ 책임을 나눠서(Runner/Service/Parser) 파일 형식이 바뀌면 Parser만 고치면 된다.
왜 처음 코드가 틀렸나?	파일 형식이 명세에 없어서 SGIS 2/5/7자리를 **내가 가정**했다. 실제 파일을 확인하지 않은 게 원인이다. 지금은 실제 파일로 확인했고 테스트 파일도 8자리로 바꿨다.
상위 지역은 왜 파일에서 안 읽고 만드나?	파일에 읍·면·동만 있어서다. 코드 앞자리로 묶고 경계는 JTS union으로 합친다.
`@Transactional`이 왜 필요한가?	저장 중 에러가 나면 전부 되돌리려고. 명세에 "오류 시 아무것도 저장 안 함"이 있다.
왜 `Region`을 common에 뒀나?	사용자 앱도 나중에 지역을 읽어야 해서.
`RuntimeException`을 상속한 이유는?	스프링 `@Transactional`은 `RuntimeException`에서 자동 롤백한다. checked 예외면 설정을 더 해야 한다.

질문	답
왜 `getInteriorPoint`인가?	구멍·섬 있는 동네에서 무게중심은 영역 밖에 떨어질 수 있다. 명세가 요구하기도 했다.
`System.exit`은 왜?	1회성 작업이라 끝나면 앱을 꺼야 해서. 이 방식은 내가 정했고 팀 확인이 필요한 부분이다.

5. 솔직하게 말해야 할 약점

Notion 페이지에 스스로 적어 둔 것들이다. 선배가 먼저 물으면 당황하지 말고 인정한다.

1. **진짜 MySQL 8.4로는 아직 못 돌렸다.** JSON 칼럼과 `DECIMAL(9,6)`은 H2(테스트용 DB)에서만 확인했다. 이 PC에 Docker와 MySQL이 없다(리뷰 R3).
2. **개정판 커밋 `865a63e`는 push 전**이라고 페이지에 적혀 있다. 그런데 같은 페이지의 `상태` 속성은 "완료", `내용`에는 "PR #100 dev 병합 완료(2026-10-20, d66cd4c)"로 적혀 있어 서로 다르다. **내일 말하기 전에 GitHub에서 PR #100의 실제 상태를 직접 확인할 것.**
3. **전국 파일 전체(3,559건)로는 돌려 보지 않았다.** 파일 전체를 메모리에 올리고 시·도 union을 하므로 느리거나 메모리를 많이 쓰는지 모른다.
4. **새 파일에서 사라진 지역은 DB에서 지워지지 않는다.** 덮어쓰기만 한다. 삭제 여부는 결정 0014의 "남은 결정"이라 구현하지 않았다.
5. **러너(Runner)에는 자동 테스트가 없다**(R2). `--file`을 안 줬을 때 에러가 나는지 보호되지 않는다.
6. **작업을 돌리면 웹 서버까지 같이 켜진다**(R1). 관리자 앱이 이미 켜져 있으면 포트가 충돌할 수 있어서 `--spring.main.web-application-type=none` 안내가 필요하다.
7. **`buffer(0)` 보정 경로**를 실제 파일에서 타 봤는지는 확인하지 못했다.
8. **꼬인 도형 검사(`isValid`)는 일부러 생략**했다. 공공 데이터는 미세 오류가 흔해서 허용했다(R5).

6. 내일 설명할 때 권장 순서

1. 한 문장 요약 + 왜 필요한가 (뒤 기능의 기준 데이터)
2. 물류센터 그림으로 5개 클래스의 역할
3. DB 표 → 엔티티 (재료)
4. Parser → Service → Runner (코드 흐름)
5. "처음에 가정이 틀려서 이렇게 고쳤다"는 개정 이야기 (솔직함이 점수를 딴다)
6. 테스트 10개로 증명
7. 약점은 먼저 꺼내서 말하기